

Sync Guide

Discovering notes and maintaining spendable state

m@rek.onl

Abstract

This volume of *The Zcash Arboretum* constructs the state machine that turns a stream of chain data into a usable shielded wallet. It assumes the *Crypto Guide*'s authenticated trees, the *Consensus Guide*'s branch-local state and reorganisation rules, the *Ironwood Guide*'s note decryption and membership equations, and the *Wallet Guide*'s account and transaction lifecycle. It explains what data to fetch, how to recognise received money, how to maintain the evidence needed to spend it, and how to undo observations when the selected history changes. It then identifies the remaining server-trust and metadata boundaries. It does not equate successful scanning with independent consensus validation.

Contents

1	What a wallet must reconstruct	3
1.1	A complete wallet record	3
1.2	Inputs and authority	3
2	Compact discovery	3
2.1	The information retained	4
2.2	Decrypting without an authentication tag	4
2.3	Incoming discovery and spend discovery	5
3	Commitment positions and witnesses	5
3.1	Positions are arithmetic	5
3.2	Empty roots and the incremental frontier	6
3.3	Retaining paths for marked leaves	6
3.4	Shards and checkpoint roots	7
4	Atomic scanning and rollback	7
4.1	The sequential state machine	7
4.2	Parallel and out-of-order work	8
4.3	Detecting and applying a reorganisation	8
4.4	Restoring from a birthday	8
5	Server trust and observable metadata	9
5.1	Four independent checks	9
5.2	What requests reveal	9
5.3	Mitigations and their limits	9
5.4	The synchronisation contract	10

1 What a wallet must reconstruct

The blockchain publishes commitments, nullifiers and encrypted note data. Their concrete constructions are in the *Ironwood Guide*, “Notes and their commitments”, “Nullifiers and authenticated existence”, and “Note delivery and recovery”. In particular, the private path of sibling hashes used to prove membership is the note’s *membership witness*. A wallet needs a more useful private view: which outputs pay its keys, what values they carry, whether they remain unspent, and which witness will allow each to be spent. These facts arise from different operations. Decryption discovers a note; commitment reconstruction checks its content; ordering determines its position; nullifier matching observes a spend; and chain validation or an explicit trust assumption establishes the branch in which those observations have meaning.

1.1 A complete wallet record

For each detected note, retain enough information to reconstruct

(pool, account, scope, note, position, creation, nullifier, spend observation).

The note includes its recipient, value, creation identifier and seed, or equivalent fully reconstructed components. Its creation record identifies block hash, height, transaction identity and Action/output index. A spend observation is branch-linked and may be absent. A note nullifier may also be unavailable to a wallet given only incoming viewing authority.

Per pool, retain the current commitment-tree size and enough tree data to produce required paths at chosen checkpoints. A *checkpoint* is a retained tree state associated with a particular block boundary. Its root and size must be bound to a branch identity, not just a height.

These records serve the *Wallet Guide*, “Fees, inputs, and change”. A detected note without a usable witness can contribute to a displayed balance while not yet being selectable for a transaction.

1.2 Inputs and authority

A sequential scanner needs:

1. account viewing keys and the external/internal scopes to search;
2. a starting branch point and its pool tree states;
3. complete relevant commitment and nullifier sequences after that point, in their protocol order;
4. output data sufficient for note discovery;
5. a way to decide whether the supplied branch and its data are authentic, or an explicit trust decision about their source.

The last item cannot be supplied by the first four. Matching an encrypted output to one’s key does not establish that the output was mined or that the server showed every spend.

The normative and implementation reference baseline is zips commit 69610984, especially the note-decryption rules and ZIP 307, and librustzcash commit e30517e4, especially `zcash_client_backend/src/scanning.rs`, `src/data_api/chain.rs` within that crate, and `proto/compact_formats.proto`. The compact schema’s inclusion of Ironwood fields is capability evidence. It is not proof that their pool rules have activated on a particular network.

2 Compact discovery

A full transaction contains proofs and signatures that a basic key-discovery scan does not need. A *compact block* is a service representation retaining selected discovery and tree-maintenance data. It is not another consensus block format.

2.1 The information retained

A compact Orchard-protocol Action carries:

Field	Bytes	Purpose
Nullifier	32	Spend detection; creation identifier of the paired output.
Extracted commitment	32	Output reconstruction and tree insertion.
Ephemeral key	32	Incoming key agreement.
Ciphertext prefix	52	Lead byte, diversifier, value and note seed.

These fields total 148 payload bytes before service encoding overhead. The omitted incoming ciphertext contains 512 encrypted memo bytes and the 16-byte authentication tag. The 80-byte outgoing ciphertext, net value commitment, randomised key, proof and signatures are also not part of this compact Action.

A compact transaction supplies its original transaction index and identifier, plus the relevant pool lists. A compact block supplies height, hash, parent hash and pool tree-size metadata where supported. Transaction indices can have gaps when the service omits transactions with no requested relevant components. The retained lists still have to preserve the exact order of commitments and nullifiers.

The specified Ironwood extension uses a distinct Action list and tree size. Its Action payload has the same shape. Do not merge its commitments with the original Orchard list: the two pools have separate position spaces and roots even though their receivers and primitive hashes are shared.

The service uses *Protocol Buffers*, a message encoding with numbered fields and prescribed default values. It can represent an absent field by its default zero or empty value. A scanner must interpret presence and metadata semantics according to the service contract; an omitted tree size is not automatically an authenticated assertion that the pool is empty. The compact block’s optional header is not guaranteed to be populated. The baseline schema explicitly describes normal compact headers as empty.

2.2 Decrypting without an authentication tag

First parse the Action’s field and point encodings canonically and enforce the required ephemeral-point conditions. Only then, for each incoming scalar ivk , compute the shared point $[ivk]epk$ and the KDF from the *Ironwood Guide*, “Note delivery and recovery”. The first 52 ciphertext bytes use the same ChaCha20 keystream prefix as the full encrypted note. Invert that prefix to obtain a candidate

$$(\ell, d, v, rseed).$$

There is no Poly1305 tag in the compact data, so this is not a successful authenticated-encryption-with-associated-data (AEAD) opening. Correctness is tested by reconstructing the note:

1. Require canonical outer field and point encodings and the appropriate ephemeral-point conditions.
2. Check that lead byte ℓ is allowed for the pool’s note format.
3. Set ρ to this Action’s nullifier. Derive g_d from d and $pk_d = [ivk]g_d$.
4. Derive esk and ψ , then the correct version’s rcm . Require the derived ephemeral point to match the supplied one.
5. Recompute the commitment and compare its extraction with the Action’s cmx .

Only a candidate satisfying those checks becomes a detected note. The security argument is the note-reconstruction contract, not the missing AEAD tag. When the wallet later retrieves the full transaction, it performs full ciphertext authentication and checks that the retrieved output belongs to the same committed transaction context.

A malformed compact record is not a well-formed record that merely decrypts to somebody else's key. A production scanner must distinguish structural failure from a harmless no-match result. It must not silently omit a malformed commitment and shift every subsequent note position. If the service does not supply trustworthy complete data, the safe response is to reject or repair the range, not mark it completely scanned.

2.3 Incoming discovery and spend discovery

For k incoming scopes and m output records, straightforward discovery performs km candidate key agreements and reconstruction attempts. Batching and prepared-key operations can reduce cost without changing which key/output pairs must be considered. Deriving many diversified addresses under one scope does not multiply k .

For an account with a full viewing key, a detected note's nullifier is computed by the *Ironwood Guide*, "Nullifiers and authenticated existence". Match that value against all observed nullifiers in the same pool. A match means the note was consumed on the supplied branch. Its Action pairing does not tell a recipient which input financed its output: the incoming nullifier also serves as the output's creation identifier, but the old commitment remains private.

An incoming-only key lacks the nullifier derivation capability. It can discover received value without independently reconstructing ordinary spentness for those notes. Exporting an incoming key and calling the result a complete unspent balance would promise more than that capability permits.

3 Commitment positions and witnesses

The proof needs a path to an eligible root. The encrypted note does not contain that path or its position. They arise from the ordered public commitment history.

3.1 Positions are arithmetic

Suppose a pool contains n commitments before a block, and the block appends c commitments. Their positions are

$$n, n + 1, \dots, n + c - 1, \quad n_{\text{after}} = n + c.$$

Every Action contributes its output commitment, including a zero-valued dummy output. Counting only notes detected by this wallet is incorrect.

For a transaction with several Actions, add the count from all previous transactions' same-pool Actions to n , then add the Action's within-list index. If reliable end-of-block metadata gives n_{after} , one can derive $n = n_{\text{after}} - c$, checking underflow and continuity with known states. This arithmetic locates leaves; it does not authenticate the metadata or prove that c includes every required commitment.

For the shared payment, suppose the pool had 11 leaves before the transaction. Its payment and change occupy positions 11 and 12 in the chosen Action order, even if Bob discovers only the first and Alice only the second. A preceding transaction's commitments would shift both positions by its count. A reorganisation can change these positions.

3.2 Empty roots and the incremental frontier

Use the concrete height-indexed hash H_j from the *Ironwood Guide*, “Nullifiers and authenticated existence”. Let $e_0 = 2$ be the empty-leaf sentinel and define

$$e_{j+1} = H_j(e_j, e_j) \quad (0 \leq j < 32).$$

These are the roots of completely empty subtrees of increasing height.

A *frontier* stores the roots of completed left subtrees needed to continue appending. For $n < 2^{32}$ leaves, its occupied heights are the set bits of n . At height j , the retained value F_j covers the corresponding completed subtree in the ordered prefix. This is binary carrying with subtree hashes instead of digits.

To append a leaf L , first require $n < 2^{32}$. Set $x = L$, $j = 0$ and $t = n$. While t is odd, combine $x \leftarrow H_j(F_j, x)$, remove F_j , set $t \leftarrow \lfloor t/2 \rfloor$ and increment j . When t is even, store $F_j = x$ and increment the original count. At the exact full capacity, retain the resulting height-32 root as a terminal full-tree state; there is no next append.

Example 3.1 (A binary carry). For $n = 11 = 8 + 2 + 1$, frontier heights are 3, 1, 0. Appending the twelfth leaf first joins the old height-zero subtree, then joins the old height-one subtree, producing a new height-two subtree. The frontier heights become 3, 2, matching $12 = 8 + 4$. No earlier leaf is rehashed.

For a nonfull tree, recover the padded depth-32 root without storing empty leaves. Start $x = e_0$. At each height $j = 0, \dots, 31$, set

$$x \leftarrow \begin{cases} H_j(F_j, x), & \text{if bit } j \text{ of } n \text{ is } 1, \\ H_j(x, e_j), & \text{otherwise.} \end{cases}$$

The current x is the partial rightmost subtree of height j . A set bit supplies its completed left sibling; an unset bit supplies an empty right sibling. Induction on height proves that after the final step x is the root of the prefix padded to capacity.

The full frontier needs at most one field element per height and the count. It is sufficient to append and compute roots. It is *not* sufficient to reconstruct an arbitrary old leaf’s membership path: hashes deliberately discard the child information. A wallet retaining owned notes needs additional marked subtrees or witness data.

3.3 Retaining paths for marked leaves

Mark a leaf when its note is detected. A simple complete implementation retains all tree nodes; it can answer a path query by retrieving the sibling subtree root at every height. A smaller wallet retains only the nodes needed for marked leaves and checkpoints, collapsing unrelated completed subtrees to their roots.

For a marked position i and a height j , its sibling interval begins at

$$a_j = ((\lfloor i/2^j \rfloor) \text{ xor } 1)2^j$$

and covers 2^j leaves. The sibling value is the root of that interval, padded with the appropriate empty roots where the interval extends beyond the checkpoint’s populated prefix. Return these 32 values with i ; the verifier recomputes the root using the bit-directed path equation.

An append at position $p > i$ can change only the sibling subtree whose interval contains p . Its height is the highest set-bit index of $i \text{ xor } p$. Update that subtree’s partial root and retain the other siblings. A root-only collapsed subtree entirely outside all future queried paths needs no retained interior.

This retention policy has a limit. If a wallet later imports a new key and discovers an old note inside a subtree whose interior was discarded, that subtree root cannot reveal the missing path. The wallet must retrieve the necessary earlier commitments or a correctly authenticated witness. Small storage is achieved by choosing what future queries to support, not by inverting hashes.

3.4 Shards and checkpoint roots

Partition the tree into subtrees of fixed height s , called *shards*. Shard j covers $[j2^s, (j+1)2^s)$. A shard containing an owned note retains its required interior. A complete shard containing no marked note can be represented by its root. The tree above shard roots retains their ordering and connects them to the full root.

To construct a witness, concatenate the path inside the note's shard with the path of that shard root through the upper tree. The lengths are s and $32 - s$, totalling 32. For example, $s = 16$ divides the path into two sixteen-hash parts; it is a storage choice, not a new consensus tree depth.

A service may provide completed subtree roots to accelerate bootstrap. A correct root for an earlier subtree can replace replay of its interior when that interior is not needed for a marked leaf. The wallet still needs trustworthy placement, completion boundary and branch association. An unverified root supplied by the same untrusted server is a shortcut within that trust model, not an independent security proof.

A checkpoint must also retain enough of the partially filled current shard to reproduce that block's historical root. Keeping only today's root for a mutable partial shard cannot answer every older anchor query. Pruning therefore follows the chosen checkpoint and spend policy: do not remove the only information needed by a still-pending transaction or a supported rollback boundary.

4 Atomic scanning and rollback

A scan updates note records and tree state together. An *atomic* update makes all of its writes visible together or none of them. Their consistency is more important than whether the implementation processes one block or many blocks per database transaction.

4.1 The sequential state machine

Let the current scan checkpoint be $(h, b, \{n_P, F_P\}_P)$, with block hash b . For a next compact block B , perform:

1. Require expected height and parent hash, when these are supplied by the protocol, and validate all structural bounds and encodings. Compare against independently authenticated block information where available. Missing required context is not a completed scan.
2. Work in temporary state. For each pool, determine the starting tree size, checking consistency against prior state and the complete ordered commitment count.
3. Traverse compact transactions in original block order. Match nullifiers against tracked notes and collect spend observations. For every output, run candidate decryption for each relevant scope.
4. Append every commitment in pool order. A detected note receives exactly this append's position; mark that position and compute its nullifier when the supplied key capability permits.
5. Reconcile the block's collected nullifiers with detected notes, verify tree-size metadata, and derive the final roots and checkpoint. If an authenticated final root is available, compare it.
6. Atomically persist detected notes, spend observations, tree changes, transaction metadata and the new branch checkpoint. Mark the range scanned only after those writes succeed.

A rejected block or interrupted transaction leaves the previous checkpoint intact. Replaying an already committed range must be idempotent: unique block/transaction/output identities prevent duplicated notes and duplicated append effects.

The wallet's pool root after a block can be authenticated indirectly by the later history commitment described in the *Consensus Guide*, "Authenticated block history". That path must include the correct epoch, node metadata and header commitment. A root merely written in a compact message is not equivalent evidence.

4.2 Parallel and out-of-order work

Trial decryptions for distinct key/output pairs can run independently. Downloaded ranges can be cached and processed out of order if their tree positions and boundary states are already known reliably. The final database must nevertheless describe one coherent branch.

For a disconnected range $[a, b]$, do not guess its starting tree size from the last locally scanned height. Obtain a boundary state or derive the size from consistent authenticated metadata and exact counts. The scanner may know that it found a note in that range before it can produce the note's complete witness. Record this partial knowledge honestly rather than mark the whole account fully synchronised.

A scan scheduler can prioritise recent blocks to discover recent receipts, then fill older gaps. Its priority is local policy. The highest scanned height is not a sufficient progress measure: an unscanned gap can hide a note or spend. Track completed intervals and their branch associations, and expose whether the contiguous required range is complete.

The source implementation's separation of scan ranges, note discoveries and retained tree data reflects these different kinds of knowledge. The explanatory algorithm does not require a particular RPC inventory or database schema to preserve them.

4.3 Detecting and applying a reorganisation

A mismatching parent hash or a disagreement with an authenticated tip signals that the stored branch may no longer match the selected branch. Locate a common ancestor by comparing retained block identities with the replacement branch. A sequence of server replies indexed only by height must not be mistaken for proof that no reorganisation happened.

Roll back chain-derived records beyond the common ancestor:

- remove or unmine notes created only on the removed suffix;
- remove spend observations that occurred only there;
- restore pool frontiers, marked-path state, checkpoint sets and sizes;
- remove stale scanned-range assertions and mined transaction confirmations for that suffix.

Keep the user's payment intentions and retained signed transactions separate. Re-evaluate them under the *Wallet Guide*, "Tracking payment state and recovery". A removed spend may become pending again, but can also conflict or expire on the replacement branch.

Example 4.1 (Positions after rollback). Suppose a common ancestor has 11 commitments. One branch appends the two outputs of the shared payment at positions 11 and 12. The replacement branch first appends three other commitments. If the same payment is then included, its outputs occupy positions 14 and 15. The note data and commitment can be unchanged while the position and path change. Reusing the old path would be incorrect.

If the common ancestor predates retained rollback information, restart from an earlier trusted checkpoint and rescan. A retention window is a resource decision. It does not justify continuing with a mixed or unrecoverable state, and it is not a proof of consensus finality.

4.4 Restoring from a birthday

A birthday says where a required scan begins. If a restored account was created before height b , initialise its starting tree state at the chosen preceding boundary and scan all required data from there. The birthday does not reconstruct that initial root or frontier; the wallet must obtain it by replay, an authenticated checkpoint, or a declared trusted source.

A birthday later than a received note can miss the note entirely. A birthday earlier than necessary costs time and bandwidth. Neither fact reveals whether a server omitted a relevant transaction inside

the scanned range. Account enumeration and restored incoming scopes must also match the original wallet; correct block scanning under the wrong keys cannot discover the missing account.

5 Server trust and observable metadata

Running decryption locally prevents a server from learning viewing keys merely to provide compact data. It does not make the entire interaction private or independently validated.

5.1 Four independent checks

Separate these questions:

1. *Content consistency*: does a recovered note match a supplied commitment and ephemeral key?
2. *Inclusion*: do the supplied transaction and state data belong to an authenticated block or tree root?
3. *Chain validity and selection*: does that block belong to the selected valid most-work branch under the client's model?
4. *Completeness and availability*: did the client receive all data necessary to discover its notes and spends?

A commitment check answers the first. A Merkle path against a trusted root can answer a specified inclusion question. Header work alone does not validate hidden transaction proofs or account for unavailable bodies. Even correct individual inclusion proofs do not prove that the server has revealed every relevant item.

A conventional lightweight scan can trust a service for full validation and completeness while keeping keys local. State that arrangement plainly. A full-node-backed wallet can delegate those same claims to a locally controlled validator. An independently authenticating client needs additional verified data and a chain-selection argument. These are different trust configurations, not different levels of confidence in the same unauthenticated reply.

5.2 What requests reveal

A server can observe source network identifiers, connection times, requested heights, range sizes, pauses and retries. A narrow birthday range can reveal approximate wallet creation or restoration activity. A follow-up request for a full transaction immediately after compact scanning can indicate that the wallet found it relevant. A broadcast request links the submitted transaction to the requesting connection unless another communication layer prevents that observation.

The chain itself also exposes transaction sizes, Action counts, pools, public value balances and timing. Coin selection or unusual change shapes can create additional correlations. Hiding note values from the server's compact scan does not erase those public facts.

Transparent address-specific queries reveal the queried addresses to the service. A design that avoids sending shielded viewing keys but uploads transparent receiver lists has a different privacy boundary for the transparent part of the account.

5.3 Mitigations and their limits

Larger uniform range requests can reduce height-resolution leakage. Batching full-transaction retrieval can obscure which compact match triggered a request. Separating scan and broadcast services can reduce one service's direct view of both activities. Using multiple sources can reveal disagreements or reduce reliance on one operator. These techniques do not automatically provide anonymity: operators may collude, network observers may correlate timing, and all sources may repeat the same omitted data.

Fetching every full transaction avoids relevance-triggered follow-up requests but increases bandwidth. Running one's own validating node changes the server-trust boundary but not public-chain leakage. The correct choice depends on the wallet's explicit bandwidth, storage and adversary model. No mechanism in this chapter proves a universal network privacy theorem.

5.4 The synchronisation contract

A wallet is ready to construct a particular spend when the required account and scope have been scanned over the necessary coherent branch range, the input note has a verified reconstruction and unspent status under the chosen trust model, and its witness is available for the selected anchor. This is more precise than saying that the latest displayed height equals the server's tip.

The core invariant is a correspondence:

recorded note $\longleftrightarrow$ pool and branch position $\longleftrightarrow$ path to the selected anchor.

Spend observations and transaction lifecycle records must refer to that same branch. If any link is missing, retain a partial state or rescan; do not silently convert uncertainty into spendable balance.

Evidence and executable checks. The compact schema is `zcash_client_backend/proto/compact_formats.proto`. The note scan and persistence interfaces in that crate are `src/scanning.rs` and `src/data_api/chain.rs`. The database implementation is `zcash_client_sqlite/src/chain.rs`, at the pinned library commit. The generic retained-tree implementation is the `shardtree` crate, used through those interfaces. The companion `check_deployed.py` checks append occupancy, frontier roots against complete trees, every retained path and its changed sibling through all 32 appends of a depth-five toy tree, position arithmetic and rollback using a clearly labelled toy hash. It does not replace the production Sinsemilla vectors, test a deployed server for omission, or establish an independent light-client security theorem.